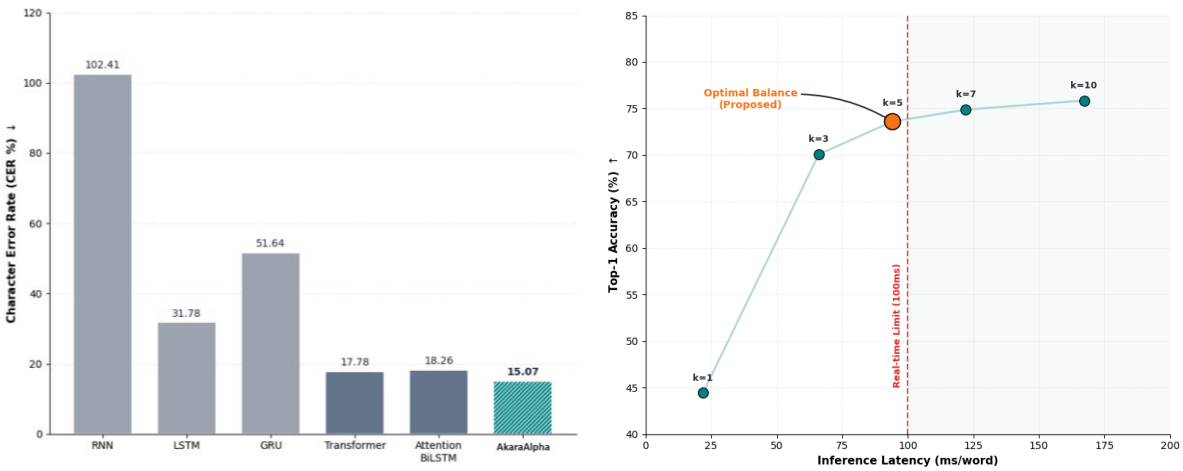
AkaraAlpha: An Efficient Romanized Khmer-Khmer Script Transliteration Model

Nhem Darayut

Abstract

In daily digital communication, Khmer speakers frequently employ informal Romanized Khmer, a practice that lacks a standardized orthographic system and presents significant phonetic ambiguity. This paper presents AkaraAlpha, an efficient deep learning model for backward transliteration, which is a process of converting these Romanized strings back into standard Khmer script. AkaraAlpha utilizes a character-level sequence-to-sequence (Seq2Seq) architecture featuring a Bidirectional Gated Recurrent Unit (BiGRU) encoder and an additive attention mechanism. To evaluate its efficacy, AkaraAlpha was benchmarked against five baseline architectures: vanilla RNN, LSTM, GRU, Transformer, and Attention-BiLSTM. Quantitative analysis reveals that AkaraAlpha achieves a superior Character Error Rate (CER) of 15.07%, outperforming the high-parameter Transformer baseline (17.78% CER). Notably, AkaraAlpha achieves this accuracy with significantly higher computational efficiency, utilizing 68% fewer parameters than the Transformer (78,705 vs. 248,337). To further enhance accuracy, we introduce a post-processing pipeline integrating beam search and dictionary-based cross-validation. This hybrid approach achieves a Top-3 word-level hit rate of 82.36%. Efficiency analysis confirms that the AkaraAlpha pipeline maintains an average inference latency of 94.15ms per word, falling within the threshold for real-time application. To promote research advancement, we open-source the model weights, training scripts, and a web-based inference application. Codes and model weights are publicly accessible at https://github.com/NDarayut/english-khmer-transliteration

Keywords: *Transliteration, Sequence-to-Sequence model, Bidirectional, Phonetic, Orthographic, Low-Resource Language*



(a) Performance Benchmark (b) System Optimization

Figure 1 | Performance benchmark and system optimization. (a) Comparison of standalone model architectures based on Character Error Rate (CER). (b) Trade-off analysis between Top-1 accuracy and inference latency across varying beam widths (k). The red dotted line denotes the 100ms real-time latency threshold, identifying k=5 as the optimal configuration for practical deployment.

TABLE OF CONTENTS

1. INTRODUCTION 3

2. LITERATURE REVIEW 3

3. METHODOLOGY 5

3.1 DATASET AND PREPROCESSING 5

3.2 BASELINE ARCHITECTURES..... 6

3.2.1 STANDARD UNIDIRECTIONAL MODELS (RNN, LSTM, AND GRU)..... 6

3.2.2 ATTENTION-BASED BIDIRECTIONAL LSTM..... 6

3.2.3 THE TRANSFORMER..... 6

3.3 PROPOSED MODEL: AKARAALPHA..... 6

3.3.1 THE ENCODER 7

3.3.2 THE DECODER 7

3.4 EXPERIMENTAL SETUP..... 7

3.5 POST-PROCESSING TECHNIQUE..... 8

4. RESULTS AND DISCUSSION..... 8

4.1 EVALUATION METRICS - CER 8

4.2 EVALUATION METRICS – TOP-1 AND TOP-K HIT ACCURACY 8

4.3 QUANTITATIVE ANALYSIS 9

4.4 QUALITATIVE ANALYSIS..... 10

5. CONCLUSION 11

REFERENCES..... 12

1. Introduction

In an increasingly interconnected digital world, the ability to bridge linguistic divides is paramount, making transliteration a critical component of Natural Language Processing (NLP) (Karimi et al., 2011; Prabhakar & Pal, 2018). According to Cambridge Dictionary, transliteration is the process of writing words using a different alphabet or writing scripts while preserving the pronunciation. There are two types of transliteration: forward, and backward. Forward transliteration is a process of mapping a word from its original native script into a foreign script (Knight & Graehl, 1998). For example, a forward transliteration of the word "ផ្ការឈូក" into a roman script is "phkarchuk". In contrast, backward transliteration reverses the process by converting the foreign script back to its original native scripts (Karimi et al., 2011).

However, the automation of accurate transliteration, particularly for non-latin scripts or language without standard romanization system, faces significant challenges. For example, the word "ខ្ញុំ" can be expressed as "khnoum" or "nh" in roman scripts. Such variability is particularly evident in digital communication and social media, where a substantial portion of Khmer speakers utilize Romanized Khmer due to its perceived speed and ease of input compared to native Khmer script. Consequently, any successful transliteration method must not only accurately infer the correct native script from these diverse and often noisy Romanized inputs but also execute this process with computational efficiency and low latency to facilitate seamless, real-time communication.

This research addresses these challenges by developing AkaraAlpha, a lightweight deep learning sequence-to-sequence (seq2seq) model optimized for efficient and accurate character-level Romanized Khmer back-transliteration. The core objective is to design a model to resolve inherent linguistic ambiguities while maintaining the low-latency performance required for real-time communication. Specifically, this study investigates three primary research questions: identifying the optimal architecture for achieving a balance between accuracy and efficiency, evaluating the efficacy of dictionary-based cross-validation in filtering invalid script transliterations, and benchmarking AkaraAlpha's performance against state-of-the-art architectures such as the Transformer. By exploring these questions, the research aims to provide a generalized solution that bridges the gap between informal Romanized phonetics and standard Khmer orthography.

The significance of this study extends to both the academic field of low-resource NLP and the practical digital inclusivity of the Khmer-speaking community. By establishing an optimal lightweight model, this work provides a methodological blueprint for transliteration in other non-standardized linguistic contexts. Furthermore, the implementation of advanced post-processing techniques, including dictionary-based validation and multi-candidate suggestion mechanisms, demonstrates a path toward enhancing the reliability and user experience of transliteration tools. Ultimately, this research facilitates more fluid cross-script communication and supports the preservation of the Khmer language in digital spaces by making native script input as accessible as its Romanized counterpart.

Our contributions can be summarized as follows:

1) We proposed AkaraAlpha, an efficient attention-based bidirectional GRU architecture that achieved the best performance compared to state-of-the-art architecture like Transformer with far less parameters.
2) We improve transliteration accuracy by introducing post-processing pipeline that utilizes dictionary cross-validation and beam search for word suggestion.
3) We open-sourced all the model weights, scripts, and training code, to promote research advancement and reproducibility.
4) We developed a web-based application and a production-ready inference pipeline to demonstrate the model's viability in real-world digital communication use cases.

2. Literature Review

Recurrent Neural Networks (RNNs) were initially developed as a pioneering architecture capable of processing sequential data by maintaining an internal "hidden state" that served as a form of memory. These networks effectively learn by sharing parameters across time steps, a critical capability for ordered inputs. However, traditional RNNs frequently suffered from the vanishing and exploding gradient problems, which made learning long-term dependencies exceptionally difficult and restricted parallelization during training, leading to slower processing (Hopfield, 1982).

Long Short-Term Memory (LSTM) networks directly addressed the critical vanishing and exploding gradient issues inherent in traditional RNNs. LSTMs overcame these limitations by incorporating a novel memory cell alongside sophisticated gating mechanisms; input, forget, and output gates, which precisely controlled information flow, allowing effective long-term dependency learning. This innovation led to LSTMs becoming highly versatile across various Natural Language Processing (NLP) tasks, including machine translation and text generation. Despite these advancements, LSTMs remained computationally intensive due leading to slower training speeds for very long sequences, and a single LSTM layer inherently processed sequences in a unidirectional manner, limiting its contextual awareness (Hochreiter & Schmidhuber, 1997).

Gated Recurrent Units (GRUs) address the computational complexity and training speed concerns of LSTMs. GRUs simplified the gating mechanism by combining the input and forget gates into an update gate and adding a reset gate, reducing the number of parameters compared to LSTMs. This simpler design still effectively captured long-term dependencies and contributed to improved convergence properties and training stability. While GRUs offered competitive performance and efficiency, they might not consistently match the peak performance of LSTMs on certain highly complex tasks and, much like LSTMs, lacked inherent parallelism, which could still slow down training (Cho et al., 2014).

For low-resource languages like Khmer, specialized statistical approaches provided initial solutions. Kak Soky et al. (2016) directly addressed Khmer's resource scarcity and the inefficiencies of manual pronunciation transcription by developing a Grapheme-to-Phoneme (G2P) model using Weighted Finite State Transducers (WFST). This research provided a robust, data-driven framework tailored for complex phonetic and orthographic rules, and they meticulously constructed a lexicon of approximately 20,000 unique Khmer words. Their WFST method achieved a significantly lower Word Error Rate (WER) of 12.98% compared to a rule-based baseline of approximately 39.59%. Nevertheless, Kak Soky et al.'s (2016) WFST model exhibited a notable limitation due to its heavy reliance on large, high-quality training datasets, which remain scarce for Khmer, hindering generalization to novel or unseen words.

Following this, Ding et al. (2018) tackled the challenges of Khmer name romanization by systematically collecting and manually annotating 7,658 grapheme-level name pairs, meticulously pre-processed to account for inherent vowels and complex phonology. They demonstrated that specialized statistical models, specifically Conditional Random Fields (CRF) and Support Vector Machines (SVM), achieved superior performance in Khmer name romanization, reaching an impressive 0.99 grapheme precision, surpassing earlier LSTM implementations and rule-based systems. However, a significant limitation of Ding et al.'s (2018) research was its exclusive focus on name romanization, intentionally overlooking the broader challenges of full vocabulary or general word transliteration for Khmer, underscoring the need for more comprehensive approaches.

To enhance the contextual understanding of recurrent networks, Schuster and Paliwal (1997) introduced Bidirectional Recurrent Neural Networks (BRNNs). This crucial architectural innovation enabled networks to process sequential input in both forward and backward directions simultaneously, thereby accessing and utilizing both past and future contextual information for each element. This comprehensive contextual understanding proved vital for tasks where element interpretation depended on surrounding context. However, Schuster and Paliwal's (1997) BRNNs inherited the fundamental drawbacks of traditional RNNs, including susceptibility to vanishing and exploding gradients, which hindered learning of very long-term dependencies.

A pivotal advancement in sequence modeling came when Bahdanau et al. (2016) proposed an additive attention mechanism within an encoder-decoder framework. This innovation enabled the decoder to dynamically focus on relevant parts of the source sequence during target prediction, effectively addressing the bottleneck limitation of fixed-size context vectors in previous Seq2Seq models. Their proposed model jointly learned to align words and translate, improving performance, particularly on long sentences. However, Bahdanau et al. 's (2016) attention mechanism was still built upon recurrent architectures, inheriting some sequential processing bottlenecks and computational overhead.

Seeking to overcome the inherent sequential processing and parallelism limitations of recurrent and attention-augmented recurrent models, Vaswani et al. (2017) subsequently proposed the Transformer

architecture. This groundbreaking model completely abandoned recurrence and convolutions in favor of multi-head self-attention mechanisms, thereby offering superior parallelization capabilities and achieving state-of-the-art performance in various NLP tasks by allowing the model to capture global dependencies more efficiently. Despite its superiority, the Transformer, however, requires a very large amount of data to achieve state-of-the-art performance, which proves quite difficult for low-resource languages like Khmer.

3. Methodology

3.1 Dataset and Preprocessing

The primary dataset for this research comprised 28,576 Romanized–Khmer word pairs (see Table 1), generously provided by Chrea Chanchhunneng (2024).Each entry within this corpus consisted of a Romanized Khmer word meticulously aligned with its corresponding standard Khmer script equivalent. An analysis of the word length distribution (see Figure 2) within this dataset revealed distinct patterns. For the Romanized Khmer words, the distribution is largely unimodal, peaking around 7-8 characters in length, with most words falling between 4 and 15 characters. In contrast, the Khmer script words exhibited a more multimodal distribution, with prominent peaks around 3-4 characters, 6-7 characters, and 8-9 characters, indicating a slightly shorter average length and more varied structural composition compared to their Romanized counterparts. This dataset formed the core training and evaluation material for the deep learning transliteration model.

To further enhance the system's accuracy and robustness, the research also leveraged the Khmer Dictionary 44k dataset, accessible via Hugging Face. This supplementary resource contained over 44,000 Khmer words. This dictionary was critically utilized as a reference resource for post-processing validation, enabling cross-checking of the model's generated outputs against a comprehensive lexicon of valid Khmer words. This validation step is designed to ensure that the transliterated outputs maintained orthographic correctness and corresponded to existing Khmer vocabulary.

Table 1 | Structure of the dataset containing Khmer Script and its corresponding Romanized Khmer

Khmer Script	Romanized Khmer
ប្រដែ	brodae
អសង្ខៃយ	aasangkheyy
នារី	neari

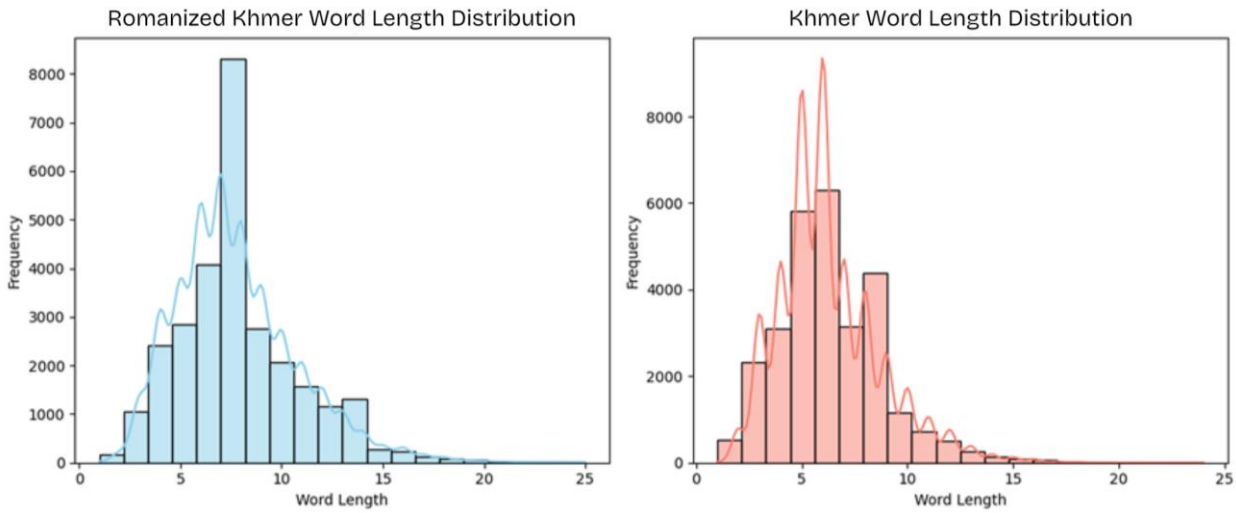


Figure 2 | Word Length Distribution of Romanized Khmer, and Khmer Script

The preprocessing pipeline is designed to prepare both the Romanized Khmer and standard Khmer text for model training. Initially, the Romanized Khmer text undergoes a cleaning phase to remove extraneous special characters (e.g., punctuation marks, symbols like !@#$%^&*;'...). Beyond that, all the Khmer text are normalized to a consistent text encoding and correct ordering of combining characters.

Following these normalization steps, both the Romanized and Khmer texts are tokenized using TensorFlow's built-in tokenization utilities. A character-level tokenization strategy is applied, rather than word-level tokenization, for several key reasons. This approach is necessitated by the task's requirement to map sequences of individual characters from a source script to a target script. More importantly, character-level tokenization enhances the model's ability to generalize to out-of-vocabulary (OOV) words and novel inputs, such as accurately transliterating foreign terms like "Facebook" into "ហ្វេសប៊ុក" which might not appear in the training corpus

3.2 Baseline Architectures

To evaluate the performance of AkaraAlpha, we implemented five distinct sequence-to-sequence (Seq2Seq) architectures as benchmark. These models were selected to represent the historical and technical evolution of sequence modelling, ranging from basic recurrent neural network to self-attention mechanism in the Transformer.

3.2.1 Standard Unidirectional Models (RNN, LSTM, and GRU)

The first set of baselines consists of standard unidirectional recurrent architectures. We implemented the Vanilla RNN to establish a performance floor. This was followed by the Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU) models. These units were included to assess how well basic gating mechanisms could handle Khmer phonetics without the aid of bidirectional context or attention. These models serve to highlight the limitations of processing Romanized strings in a single forward pass, where the model cannot "look ahead" to future phonetic cues.

3.2.2 Attention-based Bidirectional LSTM

The Attention-BiLSTM represents a significant step up in complexity and is often considered a traditional "gold standard" for Seq2Seq tasks. Unlike the unidirectional baselines, this model utilizes a bidirectional encoder to capture context from both ends of the input string. Furthermore, it incorporates a Bahdanau Attention mechanism to align input and output characters. This model is the closest functional competitor to our proposed model; however, by utilizing LSTM units (which feature an additional cell state compared to GRUs), it carries a higher parameter count and increased computational requirements.

3.2.3 The Transformer

Representing the current state-of-the-art in natural language processing, the Transformer was implemented to provide a non-recurrent benchmark. This architecture discards recurrence entirely in favor of Multi-Head Self-Attention, allowing it to model global dependencies within the character sequence simultaneously. While the Transformer is highly parallelizable and powerful, its performance in this study provides a critical comparison of whether a high-parameter, attention-only model is necessary for the relatively short sequence lengths (average 7–8 characters) found in word-level transliteration.

3.3 Proposed Model: AkaraAlpha

The proposed model is an Attention-based Bidirectional Gated Recurrent Unit (BiGRU) consisting of an encoder and decoder (see Figure 3). The encoder is responsible for extracting features from the Romanized Khmer script. The decoder is used for generating Khmer tokens based on the extracted features from the encoder in an autoregressive manner.

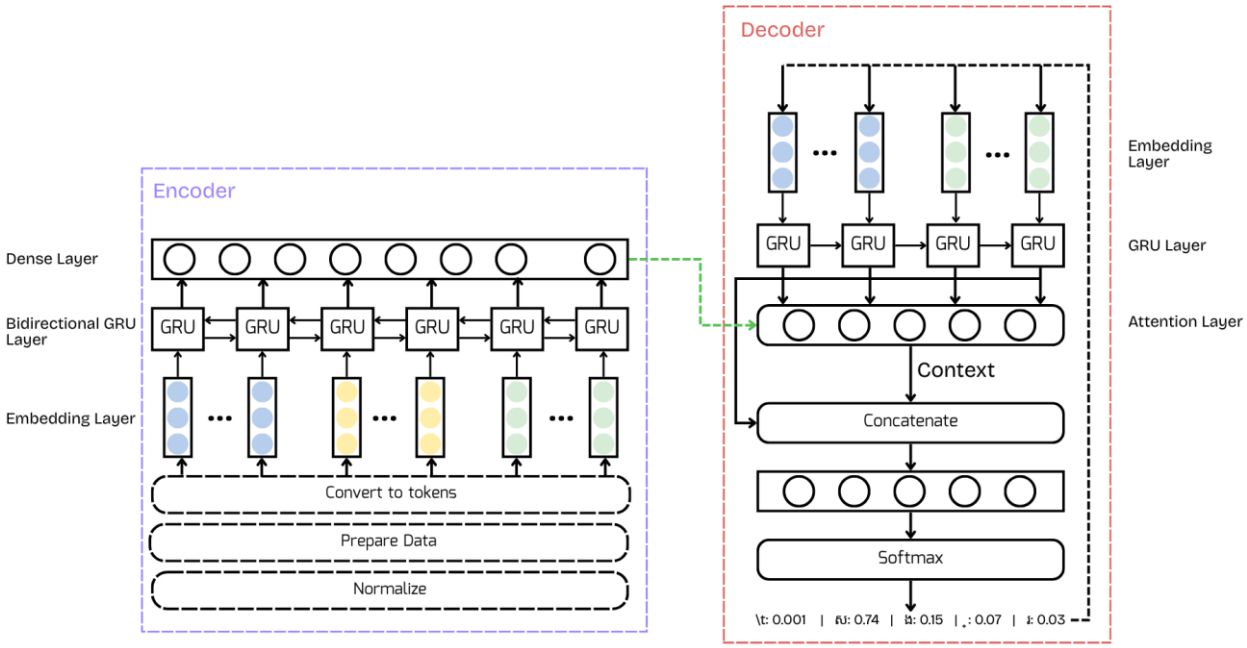


Figure 3 | The proposed Attention-based Bidirectional Gated Recurrent Unit architecture.

3.3.1 The Encoder

The encoder is used to transform the input Romanized Khmer script into a latent representation. Input characters are first mapped into a 32-dimensional space through an embedding layer. These embedding are then processed by a BiGRU layer with 64 unit per direction. This bidirectionality approach allow the model to captures phonetic dependencies from both the prefix and suffix of a word simultaneously. Because the bidirectional processing result in a 128-dimensional out, we applied a dense layer, and a linear activation function to reduce the dimension back to 64 so that it matches the decoder input shape.

3.3.2 The Decoder

The decoder is responsible for generating Khmer script character-by-character in an auto-regressive manner. It begins with its own 32-dimensional embedding layer for the target characters. A unidirectional GRU layer with 64 units maintains the internal state of the output sequence. To ensure the model correctly aligns Romanized clusters (e.g., "th") with specific Khmer characters (e.g., "ធ"), we implement an Additive Attention mechanism. This layer calculates an alignment score between the current decoder state and the encoder's projected hidden states to produce a context vector. This context vector is concatenated with the GRU output, merging the model's "local" prediction with "global" phonetic context. Finally, a Dense layer with a Softmax activation function maps this 128-dimensional combined vector to a probability distribution over the 81-character Khmer vocabulary identified in the dataset.

3.4 Experimental Setup

All baseline architectures and the proposed model were trained under a uniform configuration to ensure a fair performance comparison. The training process was conducted for 50 epochs with a batch size of 64, utilizing a 20% validation split to monitor generalization performance and prevent overfitting. Optimization was performed using the Adam optimizer coupled with a Sparse Categorical Cross-Entropy loss function. To facilitate stable convergence, we implemented a dynamic learning rate scheduler that reduced the learning rate by a factor of 0.5 whenever the validation loss reached a plateau for three consecutive epochs. All models were implemented using the Keras and TensorFlow frameworks, where strings were processed via character-level tokenization and post-padded to a fixed maximum length. Target sequences were specifically augmented with start and end control tokens to facilitate the teacher-forcing method during training.

3.5 Post-processing Technique

To enhance inference accuracy without increasing model complexity or parameter count, we implemented a post-processing pipeline (see Figure 4). This approach utilizes beam search, lexical validation, and Levenshtein distance to ensure that all generated outputs are orthographically valid and exist within the standard Khmer dictionary (Levenshtein, 1965; Ow & Morton, 1988).

The process begins with the encoder-decoder model generating three candidate transliterations using beam search (k=3). By maintaining multiple active hypotheses and calculating their cumulative probabilities, the system can identify sequences that are globally optimal rather than just locally probable. Once these candidates are generated, they are cross-referenced against a standard Khmer dictionary. If no exact match is found, the system utilizes the Levenshtein distance algorithm to identify the closest valid dictionary entry. This fallback mechanism is constrained to a maximum edit distance of two characters to ensure the suggested word remains phonetically similar to the model's original prediction.

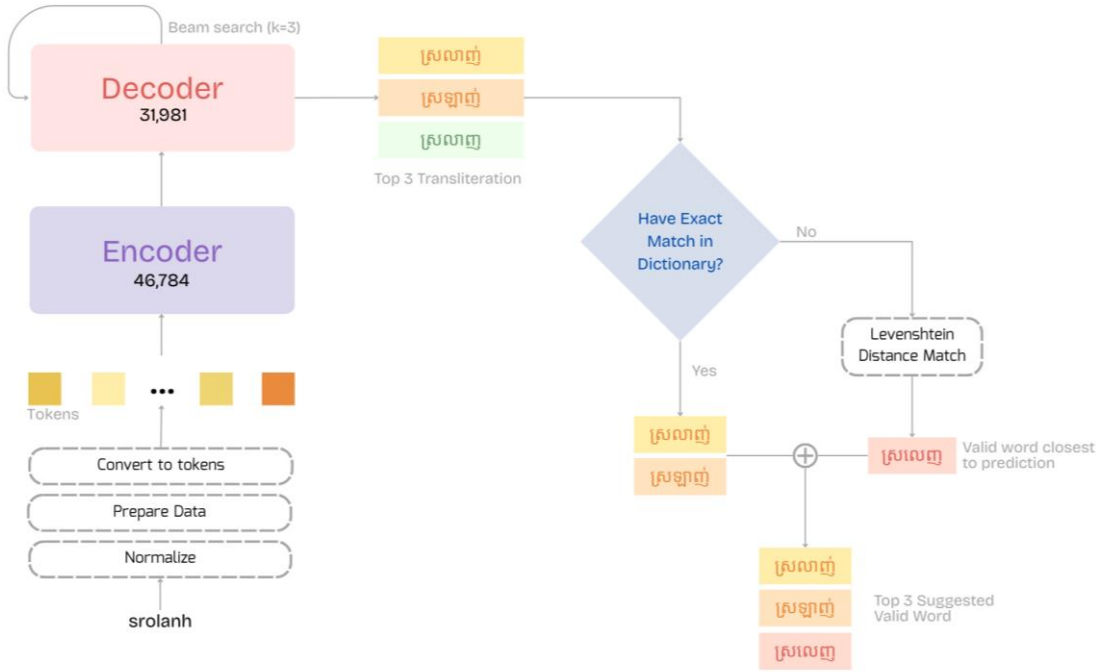


Figure 4 | End-to-End inference pipeline with post-processing technique. The input flow through the encoder-decoder model to generate 3 prediction using beam search (k=3). Following the beam search output, a lexical validation layer cross-referencing the outputs with Khmer Dictionary. The pipeline uses Levenshtein distance to recover the closest orthographically valid entries, resulting in the finals words being all valid.

4. Results and Discussion

4.1 Evaluation Metrics - CER

To evaluate the performance of the model, Character Error Rate (CER) was used. CER quantifies the number of errors at the character level, providing a direct measure of the model's accuracy in converting individual Romanized graphemes to their corresponding Khmer script. It is calculated as:

$$CER = \frac{S + D + I}{N}$$

where S represents the number of substitutions, D is the number of deletions, I is the number of insertions, and N is the total number of characters in the ground truth sequence. A lower CER indicates higher accuracy in the transliteration output.

4.2 Evaluation Metrics – Top-1 and Top-k Hit Accuracy

Additionally, to evaluate the post-processing technique, we also assess word-level accuracy by measuring the Top-1 and Top-k hit accuracy. We define Top-1 accuracy as the percentages of test samples where the highest-ranked candidates produced by the system exactly matches the ground truth:

$$Top-1 = \frac{1}{M}\sum_{i-1}^{M}(\hat{y}_{i,1} = y_i)$$

where M is the total number of test samples, y_i is the ground truth, and $\hat{y}_{i,1}$ is the number 1 ranked candidate produced by the system.

Top-k Hit accuracy measures the frequency with which the correct Khmer word appears within the list of k candidates generated by the system:

$$Top-k = \frac{1}{M}\sum_{i-1}^{M}(y_i \in \{\hat{y}_{i,1}, \hat{y}_{i,2}, \hat{y}_{i,3}, \dots, \hat{y}_{i,k}\})$$

4.3 Quantitative Analysis

The quantitative analysis was conducted based on CER and parameters count to showcased the performance as well as the size of each model. As shown in Table 2, the standard unidirectional architectures performed poorly, with the vanilla RNN failing to converge to a usable state, resulting in a CER of 102.41. While the LSTM and GRU models showed significant improvements over the basic RNN, their error rates of 31.78 and 51.64, respectively, indicate that unidirectional recurrence is insufficient for capturing the complex phonetic-to-orthographic mapping required for Khmer script. The jump in performance observed with the Attention-BiLSTM (18.26 CER) and the Transformer (17.78 CER) confirms that attention mechanisms are essential for aligning Romanized clusters with their corresponding Unicode characters.

AkaraAlpha achieved the highest accuracy among all tested architectures, reaching a benchmark CER of 15.07. This performance is particularly significant when considering model complexity; the proposed model achieved a 15.2% relative improvement in accuracy over the Transformer baseline while requiring only 78,705 parameters compared to the Transformer's 248,337. This represents a 68% reduction in model size.

The integration of the post-processing pipeline significantly improved word-level accuracy (see Table 3). Greedy decoding yielded a Top-1 accuracy of only 44.44%. However, implementing beam search at k=5 increased Top-1 accuracy to 73.60% and achieved an 83.41% Top-K hit rate. While larger beam widths (up to k=10) provided marginal accuracy gains, they resulted in diminishing returns and excessive latency (167.21ms/word). Consequently, the k=5 configuration was identified as the optimal balance, delivering high precision within the 100ms real-time latency threshold (94.15ms/word). This performance remains within the 100ms threshold identified by Miller (1968) as the critical limit for a user to perceive a system's response as instantaneous, thereby ensuring a seamless experience for real-time digital communication.

Table 2 | Model comparison on the validation set using Character Error Rates (CER %). Lower CER indicates better performance. The Lowest CER is **bold** and the second lowest is ***italicize***.

Model	CER (%)	Parameters
RNN	102.41	42,609
LSTM	31.78	58,417
GRU	51.64	46,385
Transformer	***17.78***	248,337
Attention BiLSTM	18.26	96,753
AkaraAlpha	**15.07**	78,705

Table 3 | Performance comparison of system configuration. A comparative analysis of Top-1 accuracy, Top-K hit rate, and average inference latency across greedy decoding and varying beam search widths (k)

Configuration	Top-1	Top-K Hit	Latency (ms/word)
Greedy Decoding	44.44%	44.44%	21.92
K=3	70.06%	78.72%	66.03
K=5	73.60%	83.41%	94.15
K=7	74.87%	85.60%	122.10

K=10	75.85%	87.37%	167.21

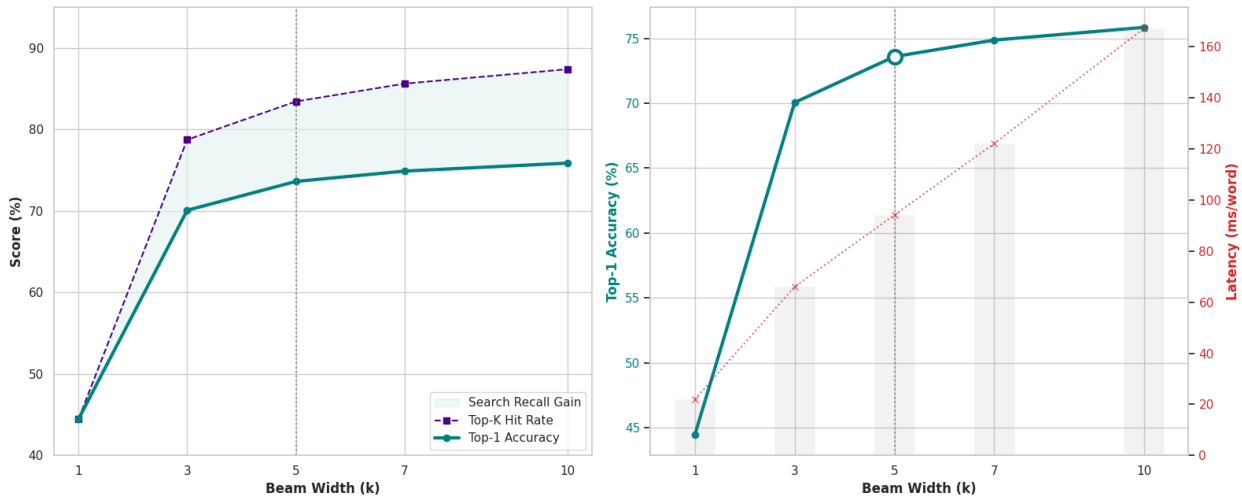


Figure 5 | Analytical trade-off between transliteration accuracy and computational overhead. (a) Search space benefit analysis illustrating the gap between primary prediction (Top-1) and system recall (Top-K). The shaded region represents the additive benefit of beam search; at the recommended *k*=5 setting, the system provides a 9.81 percentage-point (pp) recall gain over the top-1 prediction. (b) The computational efficiency frontier, highlighting the relationship between accuracy gains and inference latency. The gray bars represent the absolute magnitude of computational overhead, while the red dotted line tracks the linear latency trajectory across different beam widths. The optimal operating point is *k*=5, which achieves a 29.16% absolute accuracy increase over greedy decoding while remaining under the 100ms real-time latency threshold; beyond this point, the system exhibits diminishing returns as latency exceeds the limits of user interaction.

4.4 Qualitative Analysis

The qualitative evaluation was conducted by inspecting model outputs for four distinct test instances—"neari", "srolanh", "phkarchuk", and "jos". As shown in Table 2, the vanilla RNN consistently fails to produce linguistically meaningful results, often generating unrelated or repetitive character sequences such as "ស្រាក" or "ស្រាក់," which indicates a complete inability to learn the phonetic-to-script mapping. While the LSTM and GRU models successfully transliterate simpler words like "neari" (នារី) and "srolanh" (ស្រឡាញ់), the GRU notably stumbles on more complex phonetic structures, such as the aspirated consonant cluster in "phkarchuk," where it produces an incorrect and unrelated output.

The instance "jos" (ចុះ) serves as a critical differentiator for model robustness on short, vowel-dependent words; here, even the Attention-BiLSTM baseline fails by producing an incorrect character sequence, while the proposed model accurately identifies the terminal vowel-consonant relationship. AkaraAlpha demonstrates consistent precision across all categories, accurately rendering the complex subscript in "srolanh" and the multi-consonant cluster in "phkarchuk" (ផ្ការ៉ាឈ្ងុក).

Table 4 | Qualitative comparison across all model showing model performance across various instances of Romanized Khmer Script ranging from short to long words.

Qualitative Analysis				
Instances	neari	srolanh	phkarchuk	jos

Ground-Truth	នារី	ស្រឡាញ់	ផ្ទាល់ឈូក	ចុះ
RNN	រ៉ាក់	ស្រាក់	ស្រាក់	ទ្រាក់
LSTM	នារី	ស្រឡាញ	ផ្ទាល់ឈូក	ចុះ
GRU	នារី	ស្រឡាញ់	ផ្ទូចក្រ	ចាស់
Transformer	នារី	ស្រឡាញ	ផ្ទាល់ឈូក	ចុះ
Attention BiLSTM	នារី	ស្រឡាញ់	ផ្ទាល់ឈូក	ជោះ
AkaraAlpha	នារី	ស្រឡាញ់	ផ្ទាល់ឈូក	ចុះ

5. Conclusion

This research proposed AkaraAlpha, an efficient, and accurate Romanized Khmer back-transliteration system, addressing the lack of standardized Khmer romanization system found in informal digital communication.

The experimental results show that the proposed model achieves a superior balance between accuracy and computational efficiency compared to standard baselines. With a Character Error Rate (CER) of 15.07%, the proposed model outperformed the high-parameter Transformer baseline (17.78% CER) while utilizing 68% fewer parameters. Furthermore, we demonstrated that the integration of a post-processing pipeline using beam search and dictionary cross-validation can significantly boost the inference accuracy. The complete system achieved a final CER of 11.89% and a Top-5 word-level hit rate of 83.41%. Through a detailed trade-off analysis, we identified k=5 as the optimal configuration, delivering high-fidelity transliterations within an average latency of 94.15ms, thereby satisfying the requirements for real-time user interaction.

Despite these advancements, certain limitations remain. Specifically, the model lacks universal generalization across the diverse spectrum of Khmer romanization styles, which often vary by demographic. For instance, younger users frequently employ highly abbreviated forms, such as rendering 'ខ្ញុំ' as 'nh,' while older age groups tend to utilize more descriptive phonetic strings like 'khnhoum.' These generational differences in orthographic conventions present an ongoing challenge for character-level models in capturing the full range of informal digital communication. Future research will explore the integration of sentence-level context to resolve semantic ambiguities and the expansion of the training corpus with diverse social media datasets. By open-sourcing the model weights and inference pipeline, this work aims to provide a foundational tool for enhancing digital inclusivity and language preservation for the Khmer-speaking community.

References

Bahdanau, D., Cho, K., & Bengio, Y. (2016). *Neural Machine Translation by Jointly Learning to Align and Translate* (arXiv:1409.0473). arXiv. https://doi.org/10.48550/arXiv.1409.0473

Chanchhunneng, C. (2026). *Chhunneng/khmer-text-transliteration* [Jupyter Notebook]. https://github.com/Chhunneng/khmer-text-transliteration (Original work published 2024)

Cho, K., Merrienboer, B. van, Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014). *Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation* (arXiv:1406.1078). arXiv. https://doi.org/10.48550/arXiv.1406.1078

Ding, C., Chea, V., Utiyama, M., Sumita, E., Sam, S., & Seng, S. (2018). Statistical Khmer Name Romanization. In K. Hasida & W. P. Pa (Eds.), *Computational Linguistics* (pp. 179–190). Springer. https://doi.org/10.1007/978-981-10-8438-6_15

Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*, *9*(8), 1735–1780. https://doi.org/10.1162/neco.1997.9.8.1735

Hopfield, J. J. (1982). Neural networks and physical systems with emergent collective computational abilities. *Proceedings of the National Academy of Sciences*, *79*(8), 2554–2558. https://doi.org/10.1073/pnas.79.8.2554

Karimi, S., Scholer, F., & Turpin, A. (2011). Machine transliteration survey. *ACM Computing Surveys*, *43*(3), 1–46. https://doi.org/10.1145/1922649.1922654

Knight, K., & Graehl, J. (1998). Machine Transliteration. *Computational Linguistics*, *24*(4), 599–612.

Levenshtein, V. (1965). Binary codes capable of correcting deletions, insertions, and reversals. *Soviet Physics. Doklady*. https://www.semanticscholar.org/paper/Binary-codes-capable-of-correcting-deletions%2C-and-Levenshtein/b2f8876482c97e804bb50a5e2433881ae31d0cdd

Miller, R. B. (1968). Response time in man-computer conversational transactions. *Proceedings of the December 9-11, 1968, Fall Joint Computer Conference, Part I on - AFIPS '68 (Fall, Part I)*, 267. The December 9-11, 1968, fall joint computer conference, part I. https://doi.org/10.1145/1476589.1476628

Ow, P. S., & Morton, T. E. (1988). Filtered beam search in scheduling†. *International Journal of Production Research*, *26*(1), 35–62. https://doi.org/10.1080/00207548808947840

Prabhakar, D. K., & Pal, S. (2018). Machine transliteration and transliterated text retrieval: A survey. *Sādhanā*, *43*(6), 93. https://doi.org/10.1007/s12046-018-0828-8

Schuster, M., & Paliwal, K. K. (1997). Bidirectional recurrent neural networks. *IEEE Transactions on Signal Processing*, *45*(11), 2673–2681. https://doi.org/10.1109/78.650093

Soky, K., Lu, X., Shen, P., y, H. K., Kawai, H., z, C. V., & z, V. C. (2016, December 12). *Building WFST based Grapheme to Phoneme Conversion for Khmer*. https://www.semanticscholar.org/paper/Building-WFST-based-Grapheme-to-Phoneme-Conversion-Soky-Lu/54d6fb1601029132f3d4fbc4330414209ac1f27a

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). *Attention Is All You Need* (arXiv:1706.03762). arXiv. https://doi.org/10.48550/arXiv.1706.03762

Appendix 1-AkaraAlpha Architecture Summary

Layer	Output Shape	Param	Connected To
Input Layer (Encoder)	(None, None)	0	-
Input Layer (Decoder)	(None, None)	0	-
Embedding (Encoder)	(None, None, 32)	896	Input Layer (Encoder)
Embedding (Decoder)	(None, None, 32)	2,592	Input Layer (Decoder)
Bidirectional GRU (Encoder)	(None, None, 128)	37,632	Embedding (Encoder)
GRU (Decoder)	[(None, None, 64), (None, 64)]	18,816	Embedding (Decoder)
TimeDistributed Dense	(None, None, 64)	8,256	Bidirectional GRU (Encoder)
Additive Attention	(None, None, 64)	64	GRU (Decoder), TimeDistributed Dense
Concatenate	(None, None, 128)	0	GRU (Decoder), Additive Attention
Dense (Output)	(None, None, 81)	10,449	Concatenate

Appendix 2-Recurrent Neural Network Architecture Summary

Layer	Output Shape	Param	Connected To
Input (Encoder)	(None, None)	0	-
Embedding (Encoder)	(None, None, 32)	896	Input (Encoder)
Dropout	(None, None, 32)	0	Embedding (Encoder)
SimpleRNN 1	(None, None, 64)	6,208	Dropout
Batch Normalization	(None, None, 64)	256	SimpleRNN 1
SimpleRNN 2	[(None, 64), (None, 64)]	8,256	Batch Normalization
Input (Decoder)	(None, None)	0	-
Embedding (Decoder)	(None, None, 32)	2,592	Input (Decoder)
Dropout	(None, None, 32)	0	Embedding (Decoder)
SimpleRNN 3	(None, None, 64)	6,208	Dropout, SimpleRNN 2 (State)
Batch Normalization	(None, None, 64)	256	SimpleRNN 3
SimpleRNN 4	[(None, None, 64), (None, 64)]	8,256	Batch Normalization
Dense (Relu)	(None, None, 64)	4,160	SimpleRNN 4
Dropout	(None, None, 64)	0	Dense (Relu)
Batch Normalization	(None, None, 64)	256	Dropout
Output (Softmax)	(None, None, 81)	5,265	Batch Normalization

Appendix 3-Long Short-Term Memory Architecture Summary

Layer	Output Shape	Param	Connected To
Input Layer (Encoder)	(None, None)	0	-
Input Layer (Decoder)	(None, None)	0	-
Embedding (Encoder)	(None, None, 32)	896	Input Layer (Encoder)
Embedding (Decoder)	(None, None, 32)	2,592	Input Layer (Decoder)
LSTM (Encoder)	[(None, 64), (None, 64), (None, 64)]	24,832	Embedding (Encoder)
LSTM (Decoder)	[(None, None, 64), (None, 64), (None, 64)]	24,832	Embedding (Decoder), LSTM (Encoder) States
Dense (Output)	(None, None, 81)	5,265	LSTM (Decoder)

Appendix 4-Transformer Encoder-Decoder Architecture Summary

Layer	Output Shape	Param	Connected To
Input Layer (Encoder)	(None, None)	0	-
Input Layer (Decoder)	(None, None)	0	-
Embedding & Pos. Encoding (Encoder)	(None, None, 64)	3,392	Input Layer (Encoder)
Embedding & Pos. Encoding (Decoder)	(None, None, 64)	6,784	Input Layer (Decoder)
Transformer Encoder Block	(None, None, 64)	83,200	Embedding (Encoder)
Transformer Decoder Block	(None, None, 64)	149,696	Embedding (Decoder), Encoder Block
Dense (Output)	(None, None, 81)	5,265	Transformer Decoder Block